

## Key Takeaways

### 1 Loss Functions Shape the Gradient Landscape

MSE is the default for regression — gradient proportional to error. Cross-entropy is the default for classification — logarithmic penalty makes confident wrong answers catastrophic. When paired with sigmoid, cross-entropy's gradient simplifies to  $\hat{y} - y$  (no vanishing sigmoid derivative), while MSE's gradient includes  $\sigma'(z) \leq 0.25$  which vanishes at saturation.

### 2 Momentum Tames Oscillations, Adam Adapts Per-Parameter

Mini-batch SGD (batch size 32–256) balances speed and gradient quality. Momentum ( $\beta = 0.9$ ) builds velocity in consistent gradient directions and cancels oscillations. Adam combines momentum with per-parameter adaptive rates (RMSProp) plus bias correction. Start with Adam,  $\eta = 0.001$  — it works for 90% of problems.

### 3 Learning Rate Schedules: Start Large, End Small

Cosine annealing smoothly decays the learning rate with no hyperparameter for when to decay. Warmup (1–5% of training) prevents early instability from random gradients — critical for Transformers and large batches. The LR finder (sweep from  $10^{-7}$  to 10) identifies the optimal range in one epoch.

## Test Your Understanding

Try answering without looking at slides. Answers are on page 3.

- Q1.** (Recall) What is the gradient of binary cross-entropy loss with respect to the pre-activation  $z$  when using a sigmoid output? Why is this significant?
- Q2.** (Recall) Name the three components that Adam combines, and state the default values of its two key hyperparameters.
- Q3.** (Application) You're training a sentiment classifier and notice the loss has plateaued at epoch 15 of 100. The learning rate is constant at 0.001. What two changes would you try first?
- Q4.** (Application) Your model predicts  $\hat{y} = 0.02$  for a positive example ( $y = 1$ ). Compute the binary cross-entropy loss (approximate). Would MSE give a larger or smaller loss for the same prediction?
- Q5.** (Synthesis) A colleague argues: "Adam has momentum built in, so I don't need a learning rate schedule." Explain why this reasoning is flawed. What does a schedule provide that Adam's adaptivity does not?

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned.

### Exercise 1: MSE vs Cross-Entropy Training Comparison

Train a simple 2-layer MLP (784→128→10) on MNIST using: (a) MSE loss and (b) cross-entropy loss. Use the same architecture, optimizer (SGD, lr=0.01), and training epochs (20). Plot training loss curves for both on the same axes. Report final test accuracy for each. **Expected observation:** cross-entropy converges faster and reaches higher accuracy.

### Exercise 2: Implement SGD with Momentum from Scratch

Implement mini-batch SGD with momentum in NumPy (no autograd). Your function should take: parameters, gradients, velocity buffers, learning rate, and beta. Train on the 2D quadratic  $f(x, y) = x^2 + 10y^2$  starting from (5, 3). Plot the optimization trajectory for: (a) vanilla SGD ( $\beta = 0$ ), (b) momentum ( $\beta = 0.9$ ). **Expected observation:** momentum eliminates zigzagging.

### Exercise 3: Learning Rate Schedule Experiment

Train the same MNIST MLP with Adam (lr=0.001) under three schedules: (a) constant lr, (b) step decay (divide by 10 at epochs 10 and 15), (c) cosine annealing (min\_lr=1e-5). Run for 20 epochs. Plot all three test accuracy curves. **Expected observation:** cosine annealing reaches the best final accuracy.

## Challenge Problem

### Optimizer Showdown Dashboard

Build an **Optimizer Showdown Dashboard**. Train a 784→256→128→10 MLP on MNIST with five optimizers: SGD, SGD+Momentum, AdaGrad, RMSProp, and Adam. Use the same learning rate (0.001) and 30 epochs for all.

1. Implement the training loop with PyTorch's built-in optimizers
2. Record training loss and test accuracy after each epoch for all five
3. Create a 2×1 subplot: (top) training loss curves, (bottom) test accuracy curves
4. Add a table summarizing: epochs to reach 95% accuracy, final accuracy, total training time

**Bonus:** Add cosine annealing to the Adam run and compare against constant-lr Adam.

- **Goodfellow, Bengio & Courville (2016) — Deep Learning, Ch. 6.2** — “Output Units”  
*Explains the mathematical motivation for pairing specific losses with output activations.*
- **Ruder (2016) — An Overview of Gradient Descent Optimization Algorithms** — arXiv:1609.04747  
*Comprehensive survey of SGD variants from momentum through Adam. The best single reference for this lecture’s material.*
- **Kingma & Ba (2015) — Adam: A Method for Stochastic Optimization** — arXiv:1412.6980  
*The original paper. Sections 1–3 are accessible and explain bias correction clearly.*
- **Loshchilov & Hutter (2017) — SGDR: Stochastic Gradient Descent with Warm Restarts** — arXiv:1608.03983  
*Introduces cosine annealing with warm restarts. Clear writing, good experimental results.*

### Online Resources & Tools

- **PyTorch torch.optim** — Built-in optimizers: SGD, Adam, AdaGrad, RMSProp, plus LR schedulers
- **TensorFlow tf.keras.optimizers** — Keras API for optimizers with built-in LR decay options
- **PyTorch LR Scheduler docs** — StepLR, CosineAnnealingLR, OneCycleLR, and more
- **fastai lr\_find()** — One-line learning rate finder implementation
- **Distill.pub** — Interactive visualizations of optimization concepts (momentum, Adam)
- **Ruder’s blog post** — “An overview of gradient descent optimization algorithms” (accessible version of the survey)

### Fill the Gaps — Prerequisite Refreshers

*Felt lost on a specific concept? These resources will help.*

- **Confused about logarithms and why** —  $\log(0.01) = 4.6$ ? Review Khan Academy’s “Introduction to Logarithms” — cross-entropy lives entirely in log-space, and building intuition for log values is essential.
- **Fuzzy on the sigmoid derivative**  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ ? Re-read Session 3’s speaker notes on the sigmoid backpropagation. The key fact:  $\sigma'(z) \leq 0.25$ , which is why MSE + sigmoid causes vanishing gradients.
- **Not sure why we square gradients in AdaGrad/RMSProp/Adam?** The squared gradient tracks the *variance* of the gradient for each parameter. Dividing by its square root normalizes updates so that high-variance parameters take smaller steps. See Ruder (2016), Section 4.
- **Confused about exponential moving averages (EMA)?** EMA with decay  $\gamma$  gives weight  $\gamma^k$  to the value  $k$  steps ago. With  $\gamma = 0.9$ , the effective window is  $\approx 1/(1 - \gamma) = 10$  steps. This is the core mechanism behind both momentum and RMSProp.

### Answers — Test Your Understanding

- A1.** The gradient is  $\hat{y} - y$  — the sigmoid derivative cancels. The gradient stays proportional to the error regardless of saturation, eliminating the vanishing gradient problem of MSE + sigmoid.
- A2.** Adam combines: (1) momentum ( $\beta_1 = 0.9$ ), (2) RMSProp ( $\beta_2 = 0.999$ ), and (3) bias correction.
- A3.** (1) Add a learning rate schedule (cosine annealing or step decay) to settle into a finer minimum. (2) Try a higher initial lr (e.g., 0.01) then decay — the current lr may be too small to escape the plateau.
- A4.**  $L = -\log(0.02) \approx 3.9$ . MSE gives  $(1 - 0.02)^2 = 0.96$  — much smaller. CE penalizes confident wrong answers far more aggressively.
- A5.** Adam adapts *per-parameter* scale (second moment), but the *overall* lr controls global step size. A schedule reduces global step size over time: explore broadly early, settle precisely late. Adam doesn’t handle this.